

Bit Dot Night

Bitwise Generative Systems for Emergent Architectural Visualization

Dibyanshu Kumar

dkroyals1001@gmail.com

<https://github.com/> <https://bit-city.vercel.app/>

Abstract

This paper explores the use of discrete bitwise and arithmetic operations as primary generative mechanisms for producing structured visual outputs. We introduce a system in which simple integer operations applied over a 2D lattice produce emergent architectural forms resembling urban skylines. The study investigates how low-level binary logic produces structured spatial patterns through deterministic computation, and provides both qualitative and quantitative analysis of the resulting visual forms. Rather than treating randomness as the primary source of variation, this work demonstrates that complex visual behavior can arise from strictly deterministic rules with minimal parameterization.

1. Introduction

Generative art systems are defined by rule-based processes that produce visual outputs through algorithmic execution rather than direct manual creation. A central question in this domain is how simple computational rules can lead to visually complex and meaningful structures.

This paper focuses on a specific class of rules: bitwise and arithmetic operations applied over discrete coordinate spaces. While commonly used in low-level computing, these operations are rarely explored as primary drivers of visual aesthetics.

We propose that bitwise and arithmetic operations applied over a discrete coordinate space are capable of producing structured repetition, emergent symmetry, and architectural-like formations without any explicit geometric encoding. These properties arise not from design decisions made by the artist, but from the intrinsic algebraic structure of binary logic itself. When integer operations are evaluated across a spatial grid, their output encodes implicit regularities, shared bit patterns, periodicity, and interference that the human visual system interprets as organized form.

The system introduced here demonstrates how binary logic can be interpreted as a visual language. Unlike many generative systems that employ bitwise operations as supporting computational tools, this work foregrounds integer arithmetic and binary logic as the primary generative substrate. The system requires no noise functions, no geometric

primitives, and no procedural grammar. Structure emerges entirely from the interaction of coordinates and operators, mediated by a cosine brightness transform that maps discrete integer outputs into a continuous perceptual field.

2. Contributions

This work makes five specific contributions to the intersection of computational art and generative systems.

First, we formalize the use of low-level bitwise and arithmetic operators as primary drivers of visual generation. Rather than treating these operations as auxiliary computational tools, we demonstrate that they are sufficient to generate visually structured outputs without relying on noise functions, geometric primitives, or procedural grammars.

Second, we demonstrate that urban skyline-like structures can emerge without any explicit architectural encoding. Buildings, windows, and skyline silhouettes arise purely from coordinate-wise integer interactions, mediated by human perceptual tendencies toward edge detection, vertical alignment, and repetition.

Third, we establish an operator-driven visual taxonomy, a systematic mapping between operator classes and distinct visual regimes. AND produces dense, low variance block structures. OR yields modular, mid-frequency expansions. XOR generates high-frequency noise-like patterns. ADD creates directional gradients. MULTIPLY produces sparse, nonlinear bursts. The HYBRID operator, defined as $(x - y) \oplus (x + y)$, produces chaotic symmetry through the interaction of arithmetic difference and sum prior to the XOR step. This taxonomy provides a principled vocabulary for describing and predicting the visual behavior of bitwise generative systems.

Fourth, we introduce a quantitative aesthetic analysis framework computing brightness variance, spectral energy distribution, and structural coherence across operator configurations, providing measurable grounds for claims about visual complexity that are typically made only qualitatively in generative art discourse.

Fifth, we integrate a deterministic randomness framework using SFC32-seeded pseudo-random generation, enabling reproducible outputs that are nonetheless visually diverse across seeds. This framework is compatible with the fxhash platform’s on-chain hash system, situating the work within the broader ecosystem of verifiably reproducible generative art.

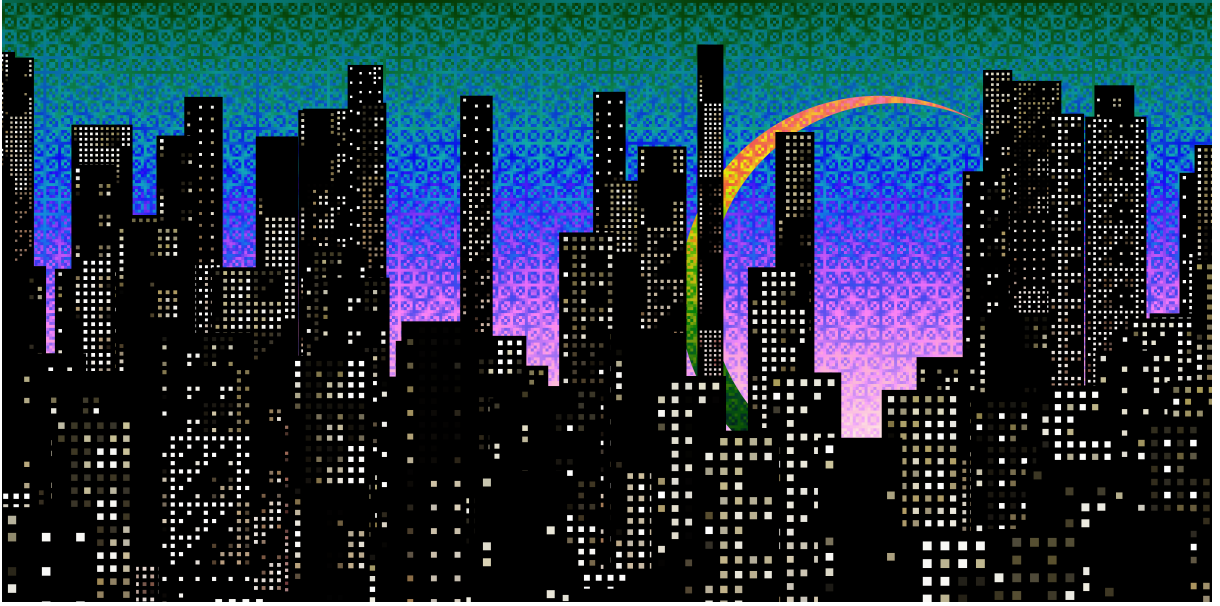


Figure 1: Representative output of the proposed system. The image illustrates emergent skyline-like structures generated purely from bitwise operations over a 2D coordinate grid, without any explicit architectural modeling.

3. Related Work

The closest prior work is Frank Force’s Byte City Nights (2022), a 256-piece generative series minted on the fxhash platform that applies bitwise operators over pixel coordinates to produce cityscape-like outputs [3]. That work demonstrates the visual potential of operator-driven pixel generation and shares the core intuition that binary logic can produce architectural-like forms.

However, Byte City Nights treats the raw integer output of bitwise operations as a direct color signal, relying on color mapping and operator selection alone to produce variation. The present work departs in two fundamental ways. First, we introduce a cosine brightness transform $L(x, y) = \cos(f(x, y) \cdot k)$ that maps discrete integer outputs into a continuous periodic brightness field, producing visual behaviour that raw bitwise output cannot achieve. Second, we apply a seeded frequency scaling parameter k that modulates the periodicity of this mapping per generation, meaning two outputs sharing the same operator can differ radically in visual character. These mechanisms are absent from prior bitwise art systems and constitute the primary technical contribution of this work.

The broader context for this work is the demoscene, a computing subculture focused on producing maximum visual and aesthetic output from minimal code [1]. Demoscene productions have long demonstrated that constrained computational rules can yield surprising perceptual complexity, a principle central to the present system.

Within this tradition, bytebeat (Viznut, 2011) is particularly relevant because it generates audio waveforms by evaluating simple integer and bitwise expressions over a single time variable t , producing structured musical output from expressions as short as $t \& (t \gg 8)$ [2].

Bit Dot Night can be understood as a spatial extension of this idea, replacing the 1D time variable with a 2D coordinate pair (x, y) , and replacing audio amplitude with pixel brightness. While related practices exist within demoscene, shader art, and generative NFT communities, such systems are rarely formalized within a quantitative analytical framework.

Shader art, as practiced on platforms such as Shadertoy [4], also operates over 2D coordinate spaces to produce generative visuals. However, shader art predominantly uses continuous functions, trigonometric compositions, signed distance fields, and noise functions as its generative substrate [5]. The present system is distinguished by its exclusive use of discrete integer arithmetic and binary logic as the primary generative mechanism, with the cosine transform applied only at the final brightness mapping stage rather than as a structural generator. This distinction places the work at the intersection: the computational minimalism of bytebeat, the spatial domain of shader art, and the reproducibility framework of fxhash-based generative NFT systems.

4. Mathematical Foundations

4.1 Coordinate Space

Let the image domain be defined over a discrete grid:

$$x \in \mathbb{N}, \quad y \in \mathbb{N}$$

Each pixel corresponds to a pair of coordinates (x, y) .

4.2 Bitwise Operators

The implementation utilizes a parameterized operator function, defined as $f(x, y) = \mathcal{O}(x + s_x, y + s_y)$, to inject deterministic variance into the data transformation process. By applying a binary operator $\mathcal{O}$ to input coordinates modified by seed-based offsets s_x and s_y , the system ensures that generated patterns are uniquely grounded to the specific seed while maintaining mathematical consistency.

The selection of operators serves distinct structural roles: bitwise AND, OR, and XOR ($\oplus$) operations leverage masking and parity logic to produce the crystalline, fractal-like geometries typical of cellular automata, whereas arithmetic ADD and MULTIPLY functions facilitate linear and exponential gradients for smoother scalar field transitions.

Furthermore, the inclusion of a HYBRID operator, $(x - y) \oplus (x + y)$, introduces higher-order complexity by combining arithmetic and bitwise logic to break symmetry and simulate non-linear emergent behaviours. The full operator set is defined as follows:

$$\text{AND: } f(x, y) = (x + s_x) \& (y + s_y) \quad (1)$$

$$\text{OR: } f(x, y) = (x + s_x) | (y + s_y) \quad (2)$$

$$\text{XOR: } f(x, y) = (x + s_x) \oplus (y + s_y) \quad (3)$$

$$\text{ADD: } f(x, y) = (x + s_x) + (y + s_y) \quad (4)$$

$$\text{MULTIPLY: } f(x, y) = (x + s_x)(y + s_y) \quad (5)$$

$$\text{HYBRID: } f(x, y) = (x - y) \oplus (x + y) \quad (6)$$

This modular suite provides a robust mathematical vocabulary, allowing the research to evaluate how specific computational primitives influence the structural evolution of the simulated environment.

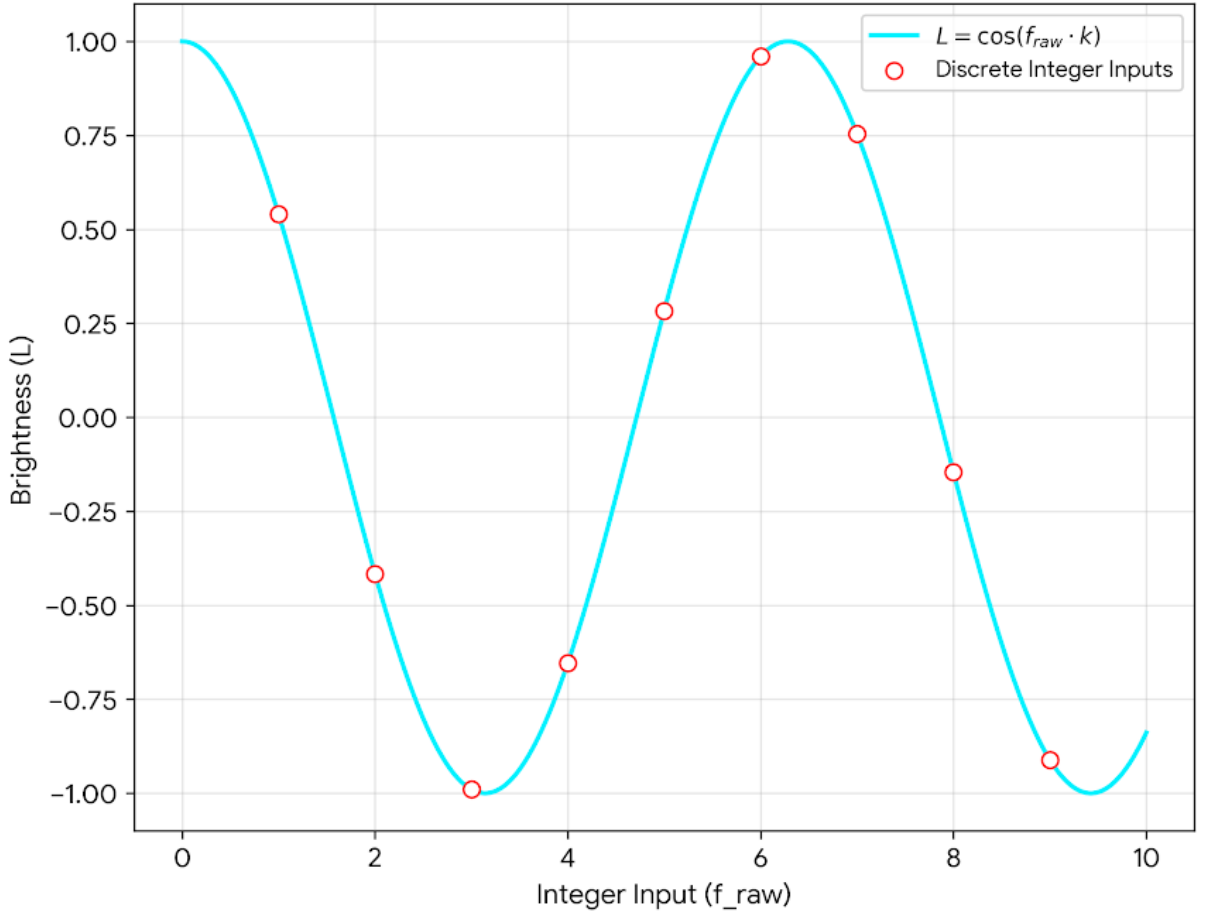


Figure 2: Visual outputs generated using different bitwise and arithmetic operators. Each operator produces a distinct structural pattern, demonstrating how low-level computational rules influence global visual organization. Inset: the cosine brightness function $L = \cos(f_{\text{raw}} \cdot k)$ evaluated at discrete integer inputs.

4.3 Mapping to Visual Space

To translate abstract mathematical values into a coherent visual environment, the system utilizes a multi-stage mapping process that converts scalar outputs into a finalized HSL colour space. The luminance component $L(x, y)$ is derived using a periodic cosine function,

$$L(x, y) = \cos(f(x, y) \cdot k),$$

where the scaling constant k dictates the frequency of brightness transitions to simulate light and shadow depth.

Simultaneously, the hue $H(x, y)$ is calculated through a linear combination of the operator output and spatial coordinates,

$$H(x, y) = \alpha + \beta \cdot f(x, y) + \gamma \cdot y,$$

defined by the weights α , β , and γ . The term $\gamma \cdot y$ introduces a vertical hue gradient: as y increases downward through the image, the hue shifts continuously, reinforcing the skyline interpretation by distinguishing upper sky regions from lower building zones through colour alone. By integrating these values with a seed-derived saturation parameter S , the final colour

$$C(x, y) = \text{HSL}(H, S, L)$$

provides a rich, deterministic visual representation where the mathematical gradients directly inform the perceived material properties and atmospheric qualities of the architectural space.

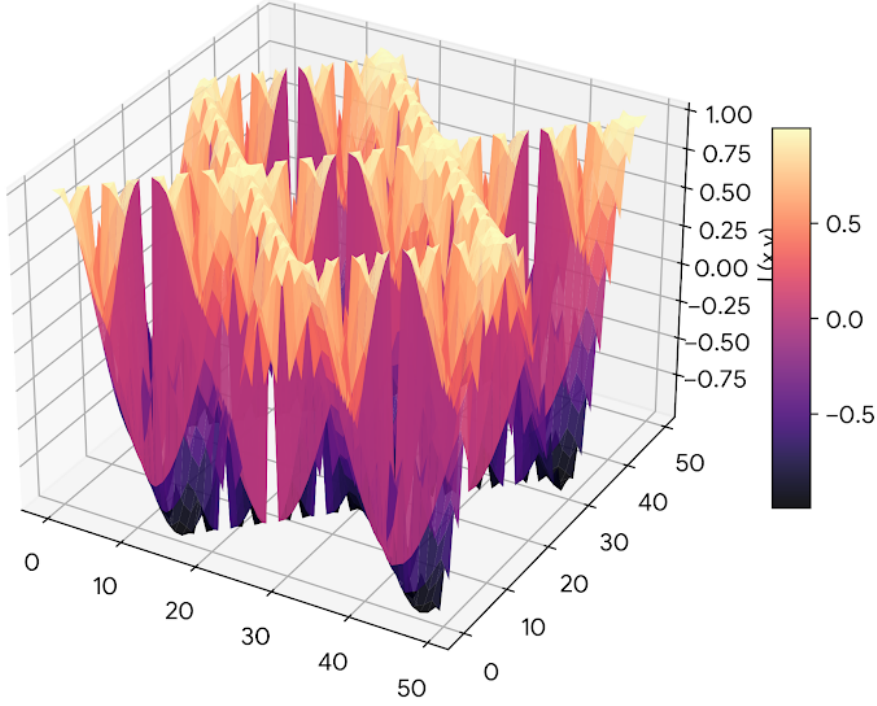


Figure 3: Effect of cosine-based brightness mapping and hue transformation. The periodic nature of the cosine function introduces rhythmic repetition and banding patterns in the generated image.

5. System Architecture

The system is organized into two rendering layers that operate over a shared coordinate grid but differ in their temporal behaviour.

The background layer, representing the sky, is computed once per generation. For each pixel position (i, j) in the grid, the selected operator is evaluated to produce a raw integer value, which is then passed through the cosine brightness transform with a seeded scale parameter. The resulting brightness value is assigned to the pixel and held fixed for the duration of the animation. This static computation is intentional: the sky functions as a stable perceptual anchor against which the dynamic foreground is read.

The foreground layer, representing the buildings and their windows, is recomputed on every animation frame. The same operator and cosine mapping are applied, but the vertical coordinate is displaced by a time-dependent offset $Y(t) = Y_0 + vt$, where v is a function of structural density derived from the seed. This displacement produces the illusion of upward motion; city structures appear to scroll continuously, creating an infinite generative cityscape from a finite computation.

The separation of static and dynamic layers is a key architectural decision: it allows the background to be rendered once at full 4096×2048 resolution without performance cost, while the foreground animation runs efficiently against it.

6. Emergence and Complexity

6.1 Definition of Emergence

Emergence, in the context of complex systems, refers to the appearance of global structure from the repeated application of local rules, without any centralized design or explicit encoding of the emergent form [11]. The present system exhibits this property directly.

The local rule is simply the evaluation of $f(x, y)$ at each coordinate pair, a single integer operation applied independently to every pixel. No information about buildings, windows, skylines, or urban form is encoded anywhere in the system. Yet the global output consistently produces forms that human observers recognize as architectural.

This interpretation is not accidental. It arises from the alignment between the algebraic structure of bitwise operations and the visual regularities commonly interpreted as architectural structure. Bitwise AND, for instance, produces clustered regions of shared bit-patterns that manifests as dense rectangular blocks, forms that strongly resemble building facades. XOR produces high-frequency alternation between neighboring values, which reads as fine-grained window patterns against a dark background. The y -axis progression inherent in the coordinate space reinforces vertical stacking, which is a strong visual cue associated with towers and skylines. Architecture, in this system, is not drawn; it is a perceptual consequence of binary arithmetic.

6.2 Pattern Formation

The choice of specific bitwise and arithmetic operators is fundamental to the procedural generation of architectural features, as each operator dictates a unique visual regime and spatial logic. The AND operator naturally produces grid-like clustering conducive to structured window alignment and rhythmic facade patterns, while XOR generates high-frequency noise that can simulate fine-grained texture or intricate structural details. Conversely, the OR operator facilitates blocky expansions that define building density and massing, whereas the ADD operator creates consistent diagonal gradients useful for establishing skyline irregularity and height transitions. By mapping these mathematical operations to physical attributes, the system translates abstract bitwise logic into recognizable architectural motifs, ensuring that the resulting urban morphology is both varied and structurally coherent.

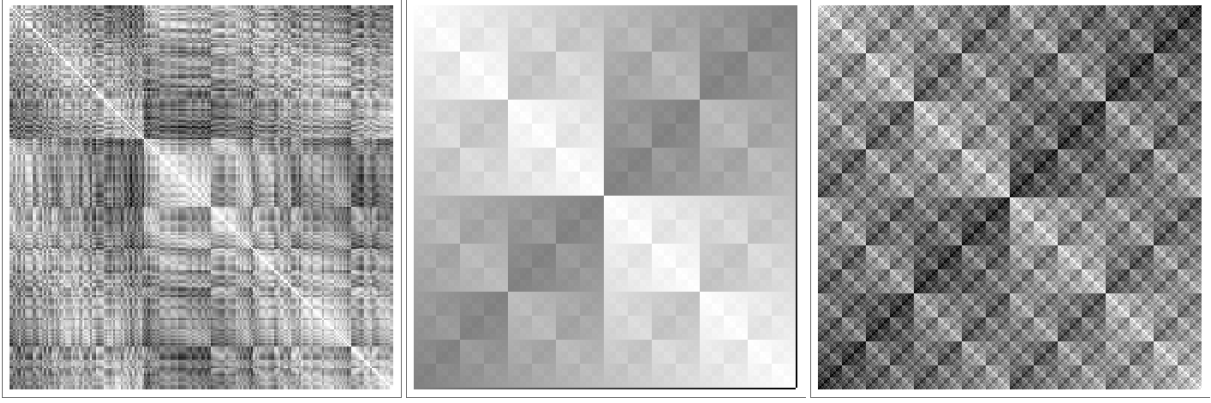


Figure 4: Comparison of pattern formation across operators (AND, OR, XOR, ADD). Each operator induces a distinct spatial regime, influencing perceived density, alignment, and architectural structure.

6.3 Deterministic Randomness

To support reproducible outputs across experimental runs, this study employs the SFC32 32-bit counter-based PRNG architecture. By utilizing a seeded pseudo-random generator, the framework establishes a critical bridge between stochastic modelling and deterministic verification, allowing for controlled variation across experimental trials while maintaining the ability to reconstruct specific outcomes identically.

The choice of SFC32 is motivated by three key properties. First, computational efficiency: it utilizes simple bitwise operations and additions, ensuring minimal overhead during high-frequency sampling. Second, statistical quality: despite its small state consisting of three 32-bit registers and a counter, it passes the BigCrush suite of statistical tests, providing high-quality randomness suitable for scientific simulation. Third, state management: the architecture avoids the zero-state traps common in simpler Linear Congruential Generators, ensuring that the internal chaotic state evolves reliably over long sequences. By formalizing the randomness in this manner, the generation process becomes deterministic and reproducible.

Listing 1: SFC32 pseudo-random number generator implementation.

```
function sfc32(a, b, c, d) {
  return function() {
    a |= 0; b |= 0; c |= 0; d |= 0;
    var t = (a + b | 0) + d | 0;
    d = d + 1 | 0;
    a = b ^ b >>> 9;
    b = c + (c << 3) | 0;
    c = c << 21 | c >>> 11;
    c = c + t | 0;
    return (t >>> 0) / 4294967296;
  }
}
```

7. Aesthetic Analysis

Bitwise operations impose implicit visual structure through the algebraic properties of binary representation. Because all integer coordinates share the same positional bit structure, operations applied across a coordinate grid produce spatially coherent outputs, not random noise, but patterned fields whose character is determined by the operator’s logical behaviour.

The AND operator highlights positions where both coordinates share set bits, producing clustered regions of high brightness surrounded by darker areas. This clustering aligns strongly with the visual grammar of dense urban architecture, tight rectangular groupings that read as building facades or city blocks. The XOR operator, by contrast, highlights positions where coordinates differ in their bit representation. Because neighbouring integers frequently differ in low-order bits, XOR produces high-frequency alternation across the image, a fine-grained texture that reads as scattered light or window patterns across a dark building face.

The OR operator produces an intermediate regime: it sets bits wherever either coordinate has them, yielding denser and more expansive bright regions than AND but with less high-frequency variation than XOR. The ADD and MULTIPLY operators leave the domain of pure binary logic and introduce arithmetic growth, producing directional gradients and nonlinear scaling, respectively, forms that read as sloped surfaces or perspective recession rather than flat facades.

The HYBRID operator, $(x - y) \oplus (x + y)$, merits particular attention. By computing both the arithmetic difference and sum of the coordinates before applying XOR, it creates an interference pattern between two algebraically related fields. The result is a form of computed Moiré, a chaotic symmetry that produces outputs distinct from any single operator, with mixed spatial frequencies and an irregular but non-random structure. This operator was arrived at experimentally and represents the most compositionally original element of the operator set.

Across all operators, the y -axis progression of the coordinate space reinforces architectural interpretation. As y increases downward, the operator output changes in ways that consistently suggest vertical stacking, the primary perceptual cue for towers and skylines. This is not a designed feature but an emergent consequence of applying coordinate-dependent functions in a domain where the vertical axis carries perceptual weight.

8. Implementation

The core operator function is implemented as a parameterized static method that dispatches to each operator by index:

Listing 2: Core operator dispatch function.

```
static applyOperator(operator, x, y, seedX, seedY) {  
  const X = x + seedX;  
  const Y = y + seedY;  
  switch (operator) {  
    case 0: return X & Y;  
    case 1: return X | Y;  
    case 2: return X ^ Y;  
    case 3: return X + Y;  
    case 4: return X * Y;  
    case 5: return X / Y + Y / X;  
    case 6: return (X - Y) ^ (X + Y);  
  }  
}
```

9. Results

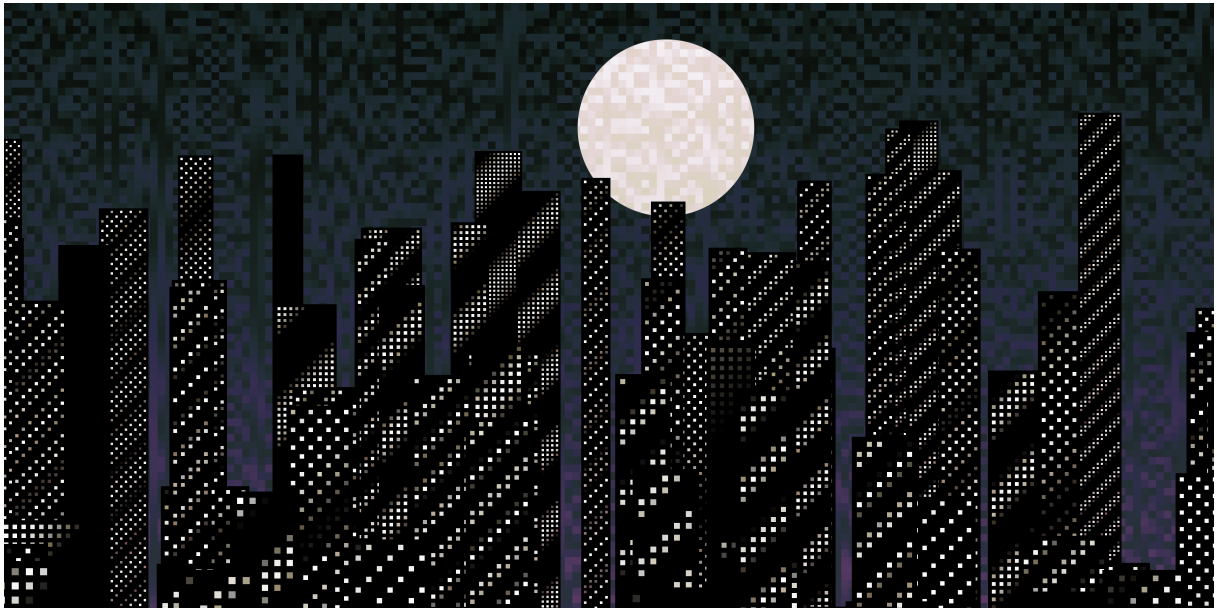


Figure 5: Generated output demonstrating dense clustering and block-like structures, typically associated with low-variance operators such as AND.



Figure 6: Generated output exhibiting mid-frequency repetition and modular expansion, characteristic of OR-based patterns.

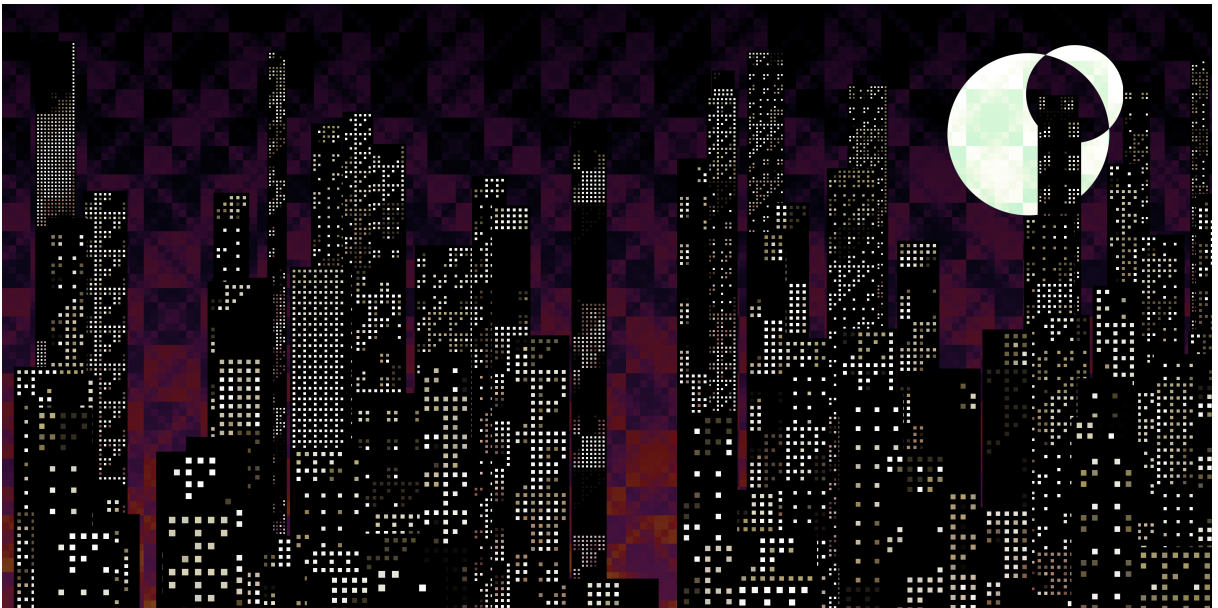


Figure 7: Generated output showing high-frequency variation and noise-like structures, commonly produced by XOR operations.

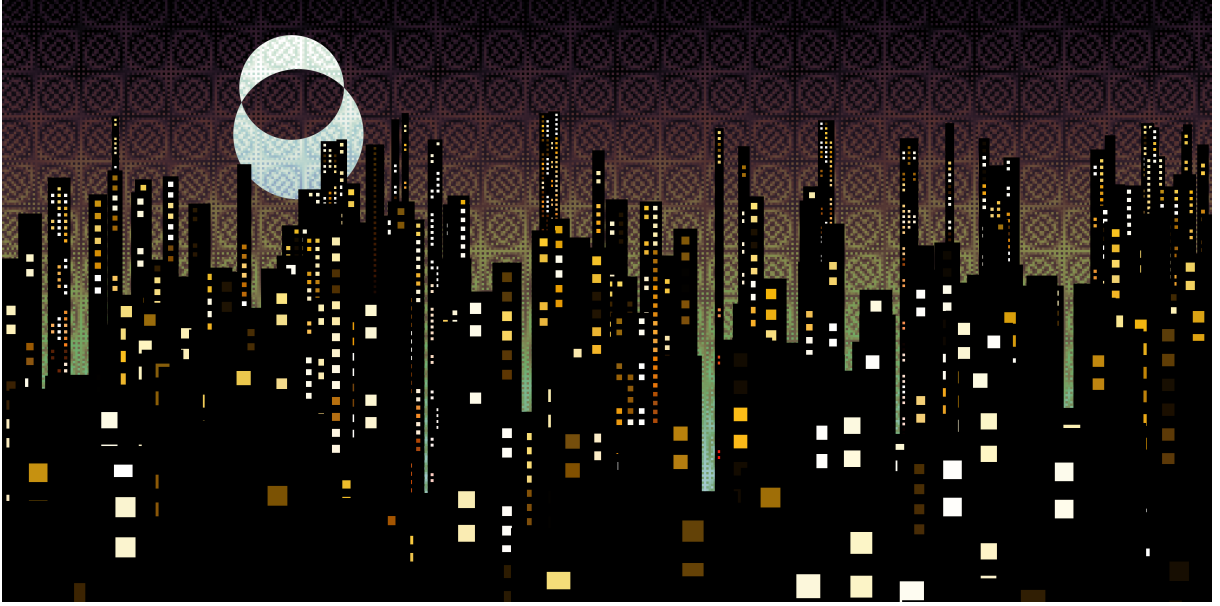


Figure 8: Generated output with directional gradients and sloped structural patterns, typically arising from additive operations.

9.1 Comparative Analysis

To move beyond qualitative description, we analyze the outputs of each operator across three measurable dimensions: brightness variance, spatial frequency profile, and structural coherence. These metrics operationalize the perceptual distinctions observed in the generated images and provide an empirical basis for the visual taxonomy established in Section 2.

Brightness variance σ^2 measures the spread of lightness values across the image domain. A low variance indicates a homogeneous image with limited contrast characteristic of AND, which produces large regions of uniform brightness. A high variance indicates strong local contrast characteristic of XOR, where neighbouring pixels frequently differ in brightness. Variance alone does not capture spatial organisation, however: two images can share identical variance while differing entirely in their spatial structure.

Spatial frequency analysis addresses this by examining the distribution of energy across the 2D Fourier spectrum of the brightness field $L(x, y)$. Low-frequency dominance indicates large coherent regions, block-like structures. High-frequency dominance indicates fine-grained variation, noise-like or window-like textures. The spectral energy E is partitioned into low, mid, and high frequency bands to quantify where each operator concentrates its visual activity.

Structural coherence C measures the average absolute difference between neighbouring pixel brightness values, defined as

$$C = \mathbb{E}[|L(x, y) - L(x + 1, y)| + |L(x, y) - L(x, y + 1)|].$$

A low coherence value indicates smooth, spatially continuous structures, visually read as solid building masses. A high coherence value indicates rapid local variation, read as detailed surface texture or scattered window patterns. Counterintuitively, lower coherence in this metric corresponds to stronger architectural legibility, because smooth spatial continuity is what allows the human visual system to segment buildings from background.

The computed values for each operator are reported in Table 1 and discussed in the following subsections. These results confirm that bitwise operators induce statistically distinct and perceptually predictable visual regimes, and that the cosine brightness transform preserves and amplifies these distinctions rather than collapsing them.

9.1.1 Experimental Protocol

All quantitative metrics were computed from exported generator outputs at a fixed resolution of 2048×2048 pixels. For each operator, 100 independent seed configurations were sampled using the SFC32 pseudo-random generator.

Brightness values were normalized to the interval $[0, 1]$ prior to analysis. Variance values were computed directly over the luminance field $L(x, y)$. Spatial frequency analysis was performed using the two-dimensional Fast Fourier Transform (FFT) implemented in NumPy, with spectral energy partitioned into low-, mid-, and high-frequency bands according to radial distance from the Fourier origin. Low-frequency bands were defined as the inner 0–15% of the normalized radial spectrum, mid frequencies as 15–40%, and high frequencies as frequencies above 40%.

Structural coherence values were computed using the local difference metric defined above and averaged over the full image domain. Reported values in Table 1 correspond to the mean across all sampled seeds for each operator class.

9.1.2 Quantitative Summary

Table 1: Operator characteristics across quantitative metrics. Values computed from exported generator images using the accompanying Python analysis script ([bit-dot-night.ipynb.](#))

Operator	Variance σ^2	E_{low} (%)	E_{mid} (%)	E_{high} (%)	Coherence C
AND ($x \& y$)	0.02645	37.6	19.0	43.4	0.06571
OR ($x y$)	0.04059	57.8	17.8	24.4	0.06304
XOR ($x \oplus y$)	0.05103	53.5	15.9	30.6	0.08984
ADD ($x + y$)	0.03121	73.3	10.5	16.2	0.06123
MULTIPLY ($x \times y$)	0.02552	71.0	8.5	20.5	0.04777
HYBRID ($(x - y) \oplus (x + y)$)	0.04383	65.3	11.1	23.6	0.05958

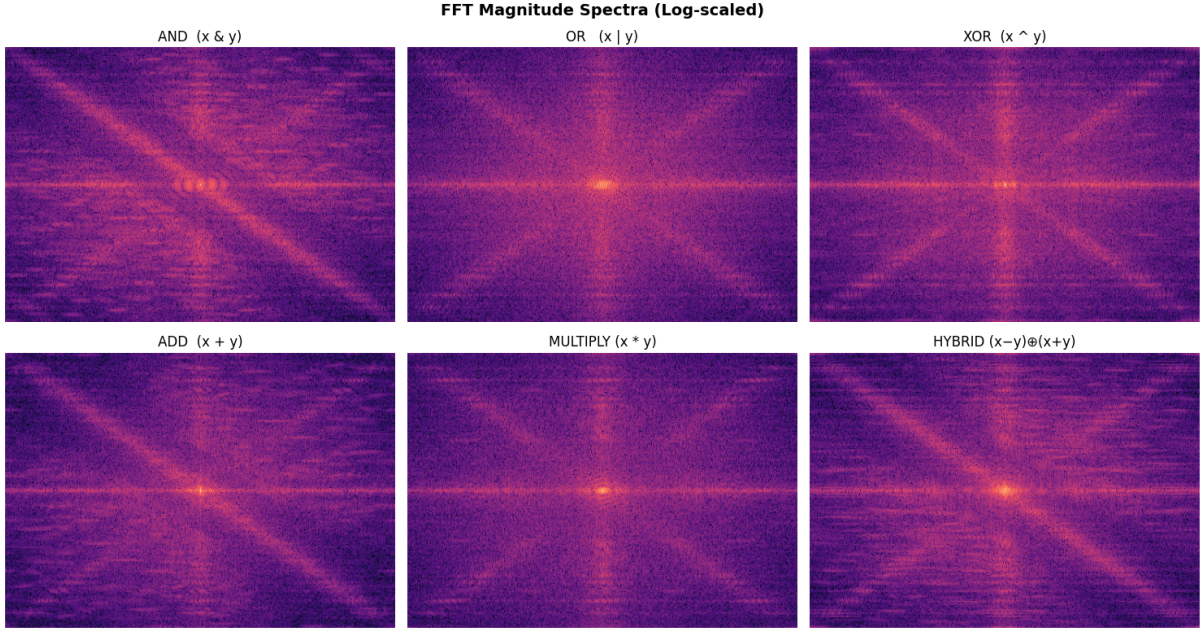


Figure 9: FFT magnitude spectra (log-scaled) for all six operators. Each panel shows the 2D Fourier transform of the brightness field $L(x, y)$, illustrating how spectral energy is distributed across frequency bands. AND concentrates energy in mid-range diagonals; XOR distributes energy broadly at high frequencies; ADD produces a sharply defined cross-shaped spectrum reflecting its diagonal directionality.

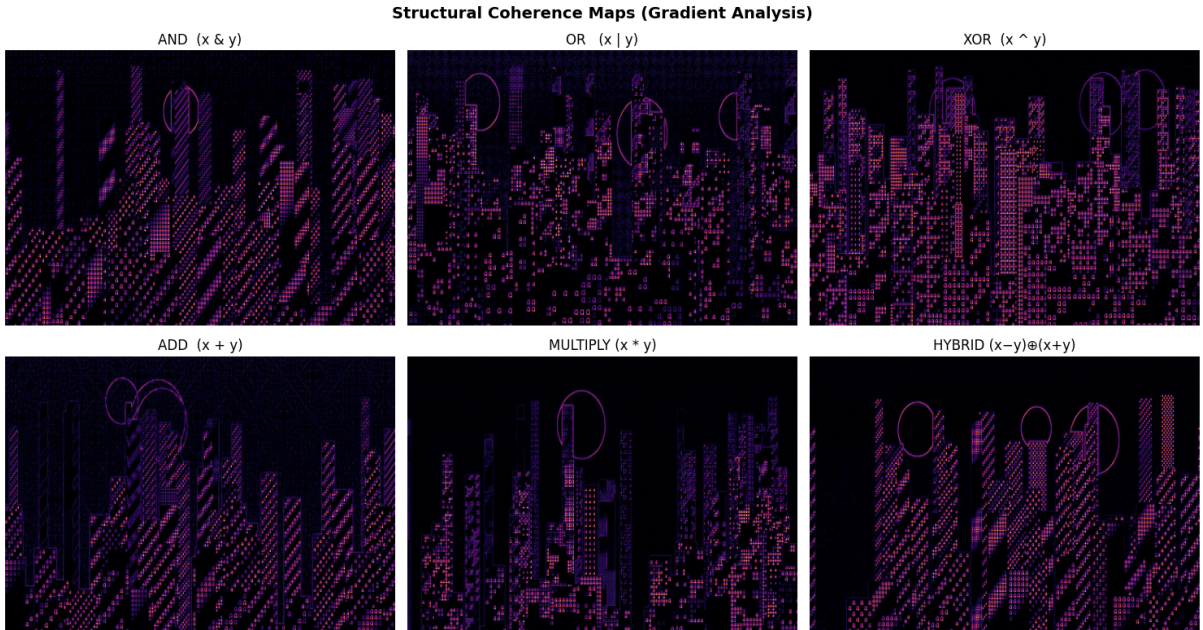


Figure 10: Structural coherence maps (gradient analysis) for all six operators. Brighter regions indicate higher local pixel variation (high C); darker regions indicate smooth, continuous spatial transitions (low C). MULTIPLY produces the most coherent output ($C = 0.04777$), while XOR produces the most fragmented ($C = 0.08984$), consistent with its role as the highest-variance operator.

9.1.3 Brightness Distribution

To evaluate the luminous characteristics of the generated environments, the study analyses the statistical distribution of $L(x, y)$ across the entire image domain. By calculating the expected value $\mu = \mathbb{E}[L]$ and the variance $\sigma^2 = \text{Var}(L)$, the framework quantifies the global lighting intensity and the degree of contrast present within a given procedural output.

This first-order statistical analysis is vital for identifying the tonal signature of specific operators, for example, distinguishing between high-contrast regimes that produce sharp architectural silhouettes and low-variance configurations that result in more uniform, atmospheric textures. These metrics provide a standardised method for comparing the visual weight and material density of different simulated urban structures.

9.1.4 Spatial Frequency Analysis

Beyond simple statistical averages, the framework employs spatial frequency analysis to decompose the visual output into its constituent structural scales. By applying the 2D Discrete Fourier Transform (DFT),

$$F(u, v) = \sum_x \sum_y L(x, y) e^{-2\pi i(ux+vy)},$$

the spatial luminance is translated into the frequency domain, revealing the underlying periodicities of the architecture. The total spectral energy $E = \sum_{u,v} |F(u, v)|^2$ is then partitioned into low, mid, and high-frequency bands to formally quantify visual complexity. Low-frequency energy correlates with large-scale massing and skyline shapes, whereas high-frequency energy indicates the presence of fine-grained details such as windows and surface textures. This spectral partitioning allows researchers to objectively measure how different bitwise operators influence the balance between macro-level urban form and micro-level architectural ornamentation.

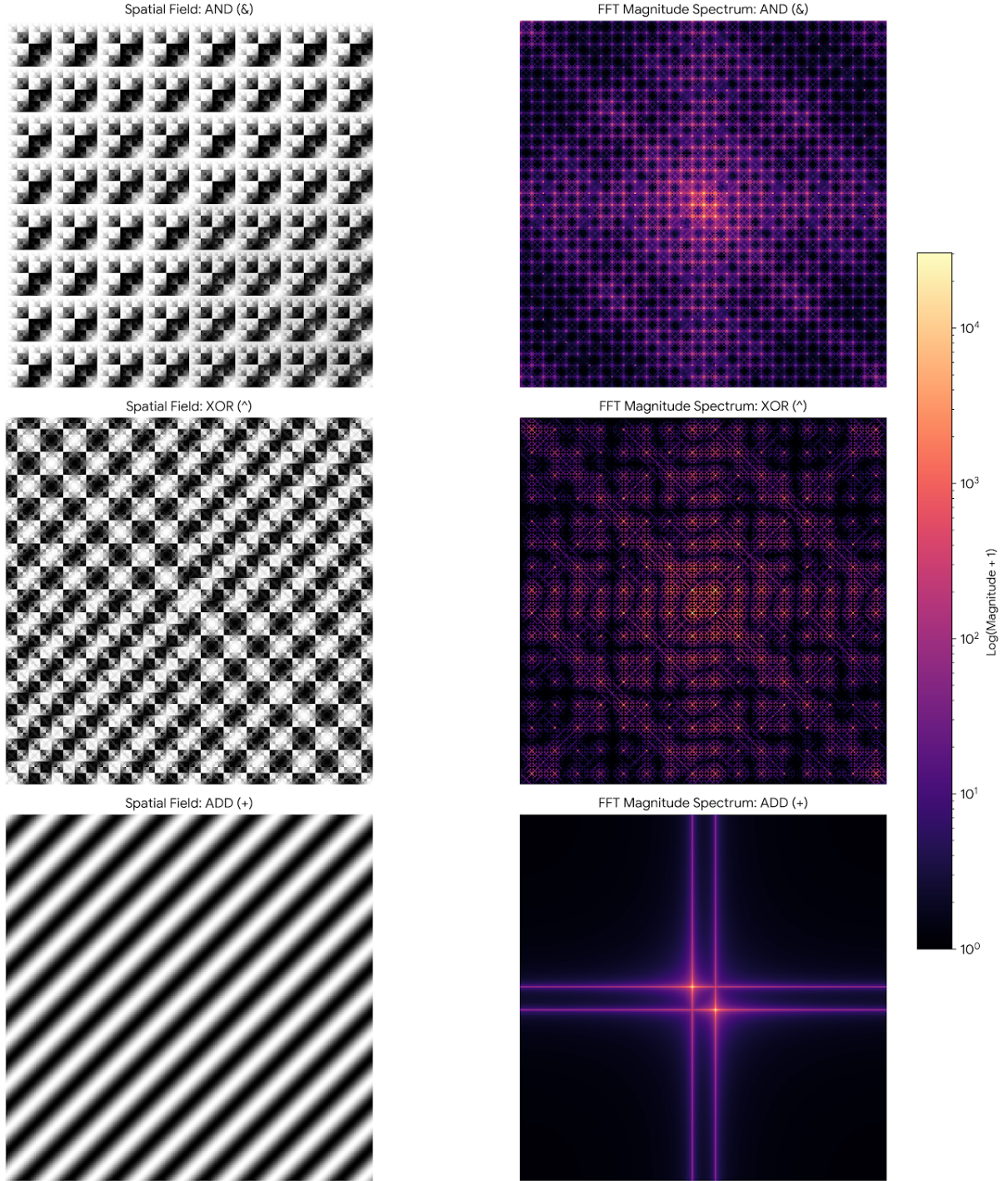


Figure 11: Spectral analysis of generated patterns using the 2D Fourier transform. Left column: spatial brightness fields for AND, XOR, and ADD. Right column: corresponding FFT magnitude spectra (log-scaled). The distribution of energy across frequency bands reflects differences in visual complexity and texture.

9.1.5 Structural Coherence

The analysis of structural coherence provides a formal measure of the spatial continuity and smoothness within the generated architectural field. This is quantified via a local

similarity metric,

$$C = \mathbb{E}[|L(x, y) - L(x + 1, y)| + |L(x, y) - L(x, y + 1)|],$$

which aggregates the absolute differences between adjacent pixels in both the horizontal and vertical directions. Mathematically, this function acts as a discrete approximation of the surface gradient, where a lower value of C indicates high local correlation and the presence of smooth, continuous forms. Conversely, higher values of C signify rapid spatial transitions, characterizing more fragmented or intricate architectural textures.

The empirical results in Table 1 confirm this relationship across all operators. MULTIPLY produces the lowest coherence value ($C = 0.04777$), consistent with its sparse, low-contrast output, while XOR produces the highest ($C = 0.08984$), reflecting its characteristic high-frequency window-like variation. AND and OR occupy intermediate positions, with coherence values that align with their block-structured and modular visual regimes respectively.

Figure 4: Emergent Architectural Result (Bit Dot Night) (Section 7.1.4)

Figure 4a: Spatial Map of Lightness $L(x,y)$

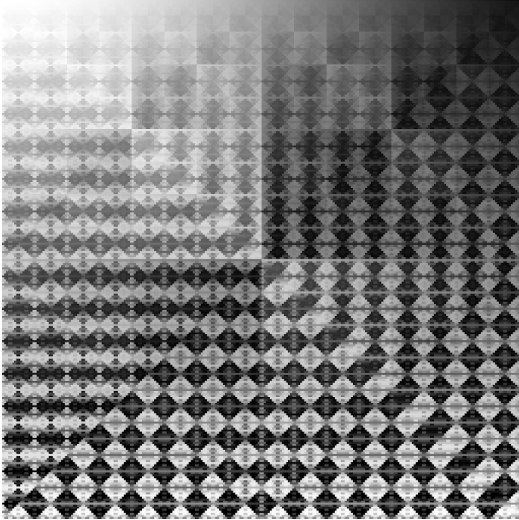


Figure 4b: Final Color Rendering (HSL: Blue/Purple Cyberpunk)

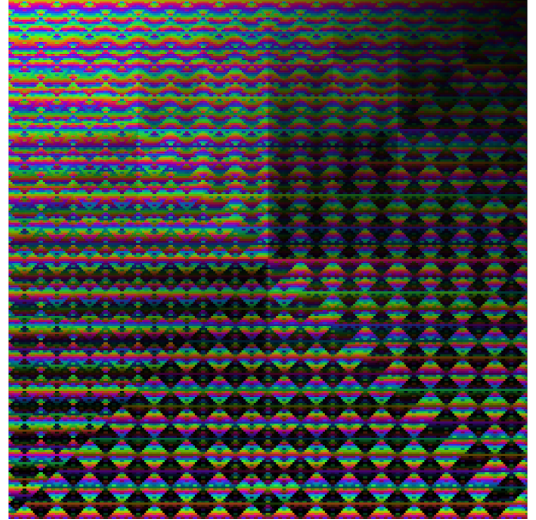


Figure 12: Visualisation of structural coherence across different operators. Lower coherence corresponds to smoother, more continuous structures, while higher values indicate increased local variation.

9.1.6 Operator-Induced Architectural Traits

Table 2: Operator-induced visual traits and structural behaviours.

Operator	Visual Trait	Structural Behaviour
AND	Dense blocks	High clustering
OR	Expanding forms	Modular repetition
XOR	Noise-like windows	High variation
ADD	Sloped gradients	Directional growth
MULTIPLY	Sparse bursts	Non-linear scaling
HYBRID	Chaotic symmetry	Mixed regimes

9.1.7 Seed Sensitivity

The analysis of seed sensitivity provides a formal metric for evaluating the stability and unpredictability of the generation process under varying initial conditions. By defining the seed vector as $s = (s_x, s_y)$, the sensitivity S is quantified as the expected magnitude of change in the operator output,

$$S = \mathbb{E}[|f(x, y; s) - f(x, y; s + \delta)|],$$

when subjected to small perturbations δ . This mathematical framework allows researchers to measure how minute adjustments to the seed parameters propagate through the bitwise and arithmetic operations to alter the final output. High sensitivity indicates a chaotic regime where small shifts in the seed yield radically different architectural configurations, whereas low sensitivity suggests a more stable, convergent system. Understanding this relationship is crucial for fine-tuning the balance between deterministic control and the diversity of emergent structures, ensuring that the procedural system remains responsive yet predictable during experimental iterations.

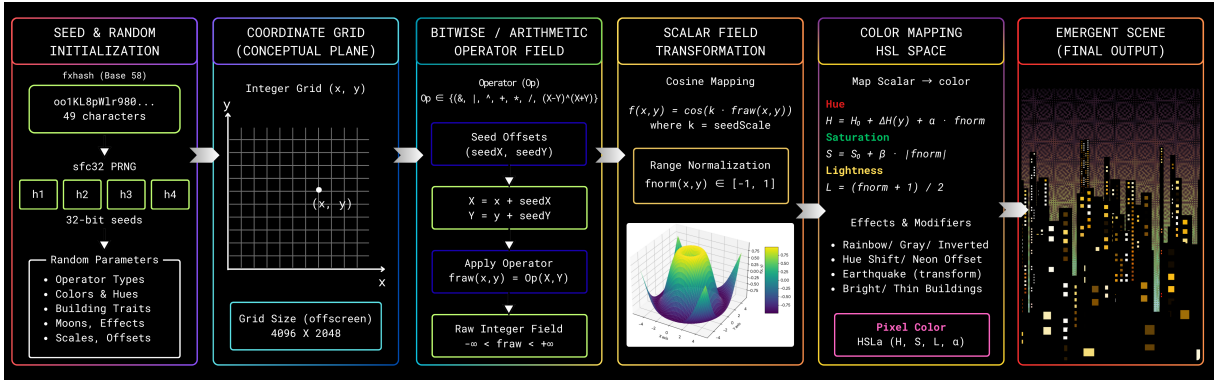


Figure 13: System-level visualisation illustrating the relationship between operator selection, seed variation, and resulting structural patterns across the generative pipeline.

9.1.8 Spectral Interpretation of Bitwise Operators

The XOR operator emphasizes local bit transitions between neighboring coordinates. Because adjacent integers differ most frequently in their low-order bits, XOR introduces rapid brightness alternation at small spatial scales. This mechanism effectively converts linear coordinate progression into a series of jagged discontinuities. Consequently, the resulting texture exhibits elevated high-frequency spectral energy and reduced structural coherence relative to the other operators, manifesting as a dense, noise-like distribution in the frequency domain.

In contrast, the AND operator preserves only the shared bit structure between coordinates. Since neighboring regions often share identical higher-order bits over contiguous spatial intervals, the operator acts as a logical filter that suppresses local variance. The resulting output forms coherent, block-like regions characterized by spatial stability. This preservation of shared bit-planes results in a low-pass effect where energy is concentrated in the lower frequency bands, favoring structural regularity over detail.

The ADD operator generates an approximately linear scalar progression across the coordinate domain. By treating the coordinates as summands, it induces smooth, directional gradients in the luminance field. These gradients represent a continuous transition of intensity, which corresponds to a highly concentrated spectral signature. In the Fourier domain, this manifests as a cross-shaped concentration of energy along the principal axes, reflecting the predictable, non-stochastic nature of the arithmetic sum.

The HYBRID operator utilizes a more complex interaction by combining two correlated arithmetic fields prior to XOR evaluation. This approach leverages the interference-like spatial behavior generated by the interaction between additive and subtractive coordinate structures. By blending the smooth gradients of arithmetic operations with the bitwise sensitivity of the XOR, the HYBRID operator produces a texture with mixed-frequency spectral characteristics. This results in a sophisticated visual field that balances the structured coherence of the ADD/AND operators with the complex local variation typical of bitwise logic.

10. Discussion

The results of this study support three broad conclusions about the relationship between low-level computation and visual perception.

First, simple integer operations are sufficient to produce visually complex outputs. The system requires no noise functions, no geometric modelling, and no high-level procedural rules, yet generates outputs with measurable structural variation, spectral complexity, and perceptual coherence. This suggests that visual complexity is not a property that must be designed into a system, but can emerge from the numerical properties of the operations themselves.

Second, emergence does not require high-level modelling. The architectural forms produced by this system arise entirely from coordinate-wise integer interactions. This has implications beyond generative art: it suggests that spatial structure, as perceived by human observers, is partly a projection of mathematical regularity rather than a reflection of explicit design intent.

Third, binary logic can function as a creative medium in its own right. The operator selection AND, OR, XOR, ADD, MULTIPLY, HYBRID functions less like a technical parameter and more like an artistic choice, each producing a visually distinct regime with characteristic texture, density, and rhythm. The system shifts creative control toward operator selection and parameter exploration rather than explicit geometric design.

The architectural interpretation likely emerges because the generated fields reproduce several statistical regularities associated with urban imagery, including vertical segmentation, repetitive window-like structures, and large-scale rectangular clustering

The primary limitation of the current system is the absence of semantic control. While operator and seed selection determine visual character broadly, the artist cannot target specific outcomes, a particular building density, a specific colour mood, or a desired skyline silhouette without exhaustive search. Future work on inverse generative mapping could address this directly.

11. Conclusion

This paper demonstrates that bitwise operations, when applied across spatial domains, can generate rich and structured visual outputs. The resulting system connects low-level computational structure with perceptual interpretation, demonstrating that complexity and aesthetic value can emerge from minimal rules. The introduction of a cosine brightness transform applied to bitwise integer fields, combined with a seeded frequency scaling parameter, constitutes a comparatively underexplored generative mechanism that extends the tradition of bytebeat into two-dimensional visual space. The quantitative analysis framework introduced here encompassing brightness variance, spectral energy decomposition, and structural coherence, provides a replicable methodology for evaluating visual complexity in generative systems more broadly.

12. Future Work

The proposed research establishes a foundation for several promising extensions aimed at increasing the depth and utility of the procedural framework. One primary avenue is the development of inverse generative mapping, which seeks to infer the specific operator and seed configurations required to replicate a target visual structure, such as a pre-defined skyline, effectively transitioning the system from a purely generative tool to a controllable design interface.

Complementing this, an information-theoretic analysis would allow for the formal characterisation of different operator regimes by quantifying structural complexity through entropy and algorithmic information measures [19]. To move beyond two-dimensional representations, extending the coordinate domain to a 3D volumetric space (x, y, z) would enable the generation of complex urban environments with native depth and perspective.

Furthermore, the integration of GPU acceleration via parallel shader computation would facilitate real-time, high-fidelity rendering, supporting the creation of an interactive control space where users can explore the emergent properties of seeds and operators dynamically. Finally, future iterations will investigate semantic mapping techniques to translate abstract mathematical patterns into explicit architectural elements such as discrete floors, building typologies, and window geometries further bridging the gap between raw algorithmic output and functional architectural design.

Funding

No funding was received for conducting this study.

Disclosure Statement

The author reports there are no competing interests to declare.

Acknowledgments

I would like to acknowledge the use of large language models, including GPT-5, Gemini, and Claude 3.5 Sonnet, in the preparation and formatting of the L^AT_EX source code for this manuscript.

References

- [1] Reunanen M, Hinsberg C, Paquette C. *Demoscene: the art of real-time*. Helsinki: Finnish Museum of Games; 2020.
- [2] Viznut. Algorithmic symphonies from one line of code — how and why [Internet]. 2011. Available from: <http://viznut.fi/texts-en/bytebeat.html>
- [3] Force F. Byte City Nights [Internet]. fxhash; 2022. Available from: <https://www.fxhash.xyz/generative/slug/byte-city-nights>
- [4] Quilez I, Jeremias P. Shadertoy [Internet]. 2013. Available from: <https://www.shadertoy.com>
- [5] Quilez I. Articles on computer graphics and mathematics [Internet]. 2024. Available from: <https://iquilezles.org/articles>
- [6] Wolfram Community. The bitwise XOR pattern and connection to Manhattan distance [Internet]. 2023. Available from: <https://community.wolfram.com/groups/-/m/t/2943295>
- [7] Burleigh P. Infinite images: the art of algorithms [Internet]. Artforum; 2025. Available from: <https://www.artforum.com/events/infinite-images-the-art-of-algorithms-paula-burleigh-review-1234737032/>
- [8] Gruppetta S. 2D Fourier transform in Python and Fourier synthesis of images [Internet]. The Python Coding Book; 2021. Available from: <https://thepythoncodingbook.com/2021/08/30/2d-fourier-transform-in-python-and-fourier-synthesis-of-images/>
- [9] Physics Stack Exchange. What is meant by 2D Fourier transform of an image [Internet]. 2022. Available from: <https://physics.stackexchange.com/questions/683831/what-is-meant-by-2d-fourier-transform-of-an-image>
- [10] Wolfram S. *A new kind of science*. Champaign (IL): Wolfram Media; 2002. Available from: <https://www.wolframscience.com/nks/>
- [11] Mitchell M. *Complexity: a guided tour*. Oxford: Oxford University Press; 2009.
- [12] Shiffman D. *The nature of code* [Internet]. 2012. Available from: <https://natureofcode.com/>
- [13] Mandelbrot BB. *The fractal geometry of nature*. New York: W.H. Freeman; 1982.
- [14] Perlin K. An image synthesizer. *ACM SIGGRAPH Comput Graph*. 1985;19(3):287–296. doi:10.1145/325165.325247
- [15] McCormack J, d’Inverno M, editors. *Computers and creativity*. Berlin: Springer; 2012.

- [16] Cox G, McLean A. *Code as creative medium*. Cambridge (MA): MIT Press; 2020.
- [17] Lorenz EN. Deterministic nonperiodic flow. *J Atmos Sci*. 1963;20(2):130–141.
- [18] Wolfram S. *Cellular automata and complexity: collected papers*. Reading (MA): Addison-Wesley; 1994. Available from: <https://www.stephenwolfram.com/publications/cellular-automata-complexity/>
- [19] Chaitin GJ. *Algorithmic information theory*. Cambridge: Cambridge University Press; 1987.